

JavaScript Execution Context - Quick Revision Notes

How JavaScript Works

3 Main Steps: Parsing → Compilation (JIT) → Execution

- Single-threaded, synchronous execution
- Creates execution contexts during execution

Lexical Environment

- **Physical location** where variables/functions are defined in code
- Determines **scope** and **scope chaining**
- Creates parent-child relationships between nested scopes

Execution Context Components

Every execution context contains:

1. **Variable Environment** - All variables and functions
2. **Scope Chain** - Links to parent scopes
3. **this value** - Context-dependent reference

Global Execution Context (GEC)

Created when: Script starts running

Creation Phase:

- Creates global object (`window` in browsers)
- Hoists variables (initialized as `undefined`) and functions
- Sets `this` = global object

Execution Phase:

- Assigns actual values to variables
- Executes code line by line



Function Execution Context (FEC)

Created when: Function is called

Creation Phase:

- Creates `arguments` object
- Sets up memory for variables/functions
- Establishes scope chain
- Sets `this` based on call pattern

Execution Phase:

- Assigns values to variables
- Executes function body
- Returns value (explicit or `undefined`)



Memory Management

Call Stack (LIFO)

- Tracks execution contexts
- **Push:** Function called → Add context
- **Pop:** Function returns → Remove context
- Limited size (stack overflow possible)

Heap

- Stores objects, arrays, functions
- Unstructured memory region
- Subject to garbage collection

Storage Rules

Data Type	Storage Location
Primitives	Stack (numbers, strings, boolean, null, undefined)
References	Heap (objects, arrays, functions)

Key Rules to Remember

1. **Single-threaded:** One command at a time
2. **Each function call** creates new execution context
3. **Function variables** can't be accessed outside their scope
4. **Inner functions** can access outer function variables (scope chain)
5. **this** value depends on how function is called
6. **Primitives** → Stack, **Objects** → Heap

7. **Call stack** has size limits

8. **Automatic** memory management via garbage collection



this Keyword Quick Reference

```
// Global context
this === window (in browsers)

// Function calls
func() → this = global object
obj.method() → this = obj
new Func() → this = new instance
```



Execution Flow Example

```
Global EC Created
↓
Global Execution Starts
↓
Function Called → Function EC Created
↓
Function Executes
↓
Function Returns → Function EC Destroyed
↓
Back to Global EC
↓
Global Execution Continues
```



Common Pitfalls

- **Hoisting confusion:** Variables are `undefined` during creation phase
- **Scope misunderstanding:** Variables accessible based on lexical environment
- **`this` binding:** Changes based on call pattern, not definition location
- **Stack overflow:** Too many nested function calls



Quick Debug Tips

- Use browser dev tools to see call stack
- `console.log(this)` to check context
- Understand hoisting for variable access issues
- Remember scope chain for variable lookup



Master these concepts for debugging, closures, async JavaScript, and performance optimization!